



UOR Foundation

<https://github.com/orgs/UOR-Foundation>

31st March 2025

Our story

Alex Flom's journey to founding UOR begins with his 11-year tenure in the U.S. Navy (Special and Expeditionary Warfare), where he tackled some of the most unforgiving technological challenges—specifically in air-gapped communications and tactical satellite systems. These environments demanded creative solutions for transmitting high volumes of data through extremely constrained channels—imagine pushing 100GB over a DSL line. This experience instilled in him a unique discipline for solving low-bandwidth, high-complexity data problems.

After his military service, [Red Hat](#) brought Alex on board to solve a long-standing issue: Kubernetes' inability to function effectively in air-gapped environments. He developed OC Mirror, a state-machine solution that enabled OpenShift to synchronize internet-based software artifacts with disconnected, secure environments. It is now part of OpenShift's core functionality.

However, OC Mirror uncovered a deeper issue: the lack of consistent metadata across software artifacts on the internet. A notable example was NVIDIA's CUDA drivers, which failed to publish necessary metadata in Kubernetes operator channels, causing painful reliance on institutional memory. This limitation revealed a broader problem—a fragmented, inaccessible internet of software and data artifacts.

Driven to solve this, Alex left Red Hat to explore more fundamental solutions. What followed was a period of intense R&D that led to the development of a new mathematical framework, enabling lossless, traceable neural networks built from first principles—without assuming numbers, algebra, or arithmetic. This work culminated in Prime OS, a new neural architecture capable of dynamically remapping data contexts through “hot-swappable” layers, effectively creating a new standard for federated, privacy-respecting data computation.

UOR (Universal Object Reference) emerged from this vision: a foundational, open-source framework designed to unify disparate data formats while preserving semantics—essentially providing a universal coordinate system for data. It aims to solve problems like hallucination in LLMs, metadata loss, data provenance, and cross-domain interoperability—challenges that traditional ontological or flat token-based models fail to address.

Now through UOR, Alex is enabling a next-generation internet standard—one that treats data as dynamic, composable, and traceable, with powerful implications for sectors like healthcare, where privacy, interoperability, and user agency are paramount.

UOR Foundation. Unlocking human potential,
with sovereign data and knowledge graph ownership.



Product Concept

I. Context: The Problem with Software Today

Modern software development is built on fragmented abstractions, opaque data flows, and black-box components. Code is copied, logic is hidden, and data often lacks context, meaning, or provenance. Applications are built without guarantees of interpretability, composability, or trust—leading to brittle systems, user disempowerment, and knowledge silos.

As AI and distributed systems accelerate, this lack of structure and traceability undermines transparency, collaboration, and individual agency. We need a new foundation that treats logic and data as first-class, structured, interoperable objects.

II. The UOR Framework: A Unified Foundation for Reasoning and Representation

The Universal Object Reference (UOR) framework is a meta-architecture for structured computation and data representation. It introduces a topological coordinate system for encoding knowledge, software, and systems as formal, composable, and verifiable objects.

Core characteristics of UOR:

- Object-Centric Modeling: Every entity—code, data, model, function—is treated as a uniquely referenced, self-contained object.
- Topological Data Graph: Objects exist within a directed acyclic graph (DAG) where relationships are explicitly structured through typed edges.
- Canonical Representation: All inputs and outputs are resolved through deterministic transformations with full traceability.
- Coherence Enforcement: Constraints across views and modalities ensure consistency, alignment, and interpretability.

In short, UOR is a machine-verifiable structure for defining, transforming, and composing knowledge—enabling applications to be not only executable, but explainable and interoperable.

III. Application Across the Prime Stack

By embedding UOR across the full Prime product suite—Prime SDK (build), Prime AppStore (ship), and PrimeOS (run)—we transform the entire lifecycle of software development into a verifiable and sovereign process.

1. Prime SDK — Build as Structured Logic

With Prime SDK, developers construct applications using UOR-native building blocks. Rather than relying on imperative codebases or implicit state, systems are modeled as explicit circuits of logic and data.

- Components are authored as referenceable UOR objects with clear input/output semantics.
- Dependencies and transformations are explicitly declared and validated through the Cortex (UOR's reasoning layer).
- Outputs are deterministic, debuggable, and compatible across any UOR-based runtime.



This makes every system composable by design—no guesswork, no hidden state, no copy-paste code.

2. Prime AppStore — Ship with Verifiable Distribution

The Prime AppStore is not just a marketplace—it's a registry of structured, logic-bound UOR applications.

- Apps are distributed as bundles of validated UOR circuits, with canonical references for every dependency.
- Users gain visibility into the logic, data lineage, and provenance of the software they install.
- Developers publish interoperable components that can be linked, reused, or extended across projects.

This unlocks transparent distribution, composability at scale, and a shared substrate for community-built logic.

3. PrimeOS — Run with Sovereign Execution

PrimeOS acts as the UOR-native operating system—resolving, executing, and verifying UOR circuits at runtime.

- Execution is deterministic and inspectable, with full traceability of every operation and data flow.
- Data sovereignty is preserved—inputs, outputs, and intermediate state are always under user control.
- Peer-to-peer logic execution, decentralized orchestration, and modular runtimes become natively supported.

PrimeOS makes it possible to run software as a graph of truth-preserving operations, not just a stream of unverified instructions.

IV. Mission Alignment: Unlocking Human Potential

Integrating UOR into the Prime stack directly advances our mission:
to unlock human potential through sovereign data ownership and open access to knowledge.

UOR makes this possible by:

- Empowering users to own their logic and data as portable, verifiable objects
- Enabling developers to build composable, reusable systems from a shared foundation
- Providing structured, transparent access to knowledge—never hidden in black boxes
- Ensuring interoperability across domains, identities, and platforms

By structuring meaning, not just surface level ontological syntax, UOR establishes a foundation where software and intelligence are as open and sovereign as their users

